ANDROID APPLICATION SECURITY ASSESSMENT

InsecureBankv2 APK – Static Application Security Testing (SAST)

Tools Used:

Kali Linux • AAPT • APKTool • JADX

Assessment Type:

Static Application Security Testing (SAST)

Application:

InsecureBankv2

Submitted By:

Saniya Shaikh

Summary

This report presents a static application security assessment of the intentionally vulnerable InsecureBankv2 APK. The assessment begins with APK reconnaissance and structural analysis, followed by Manifest and resource analysis, native-library and signing checks, JADX decompilation, manual source-code SAST, and APKTool/Smali analysis.

The assessment identified several security weaknesses, including cleartext transmission of sensitive information, hardcoded cryptographic material, weak credential protection, password disclosure through SMS and logs, insecure storage of financial transaction data, insecure WebView handling, insufficient Content Provider access control, and easily bypassable root detection. The report also documents security-relevant configurations and areas where the available static evidence does not by itself prove exploitability.

Objective

The objective of this report is to perform a static security analysis of the InsecureBankv2 Android application using Kali Linux. The assessment examines the APK structure, AndroidManifest.xml, application resources, native libraries, APK signing information, decompiled Java source code, and Smali code using tools such as AAPT, APKTool, JADX, and command-line analysis tools.

The analysis aims to identify security weaknesses related to sensitive data transmission and storage, cryptographic implementation, password handling, logging, WebView usage, exported Android components, Content Provider access control, and client-side security controls. The findings are supported with command output and source/Smali-level evidence throughout the report.

Security Findings Summary

Finding	Component	Severity
Cleartext Transmission of Sensitive Data	DoLogin.java, DoTransfer.java, ChangePassword.java	High
Hardcoded Cryptographic Key	CryptoClass.java	High
Password Disclosure Through SMS	MyBroadCastReceiver.java	High
Weak Credential Storage Using Hardcoded Encryption	DoLogin.java, CryptoClass.java	High
Password Disclosure Through Logs	MyBroadCastReceiver.java	High
Insecure Storage of Financial Transaction Data	DoTransfer.java	High
Password Exposure Through Broadcast	ChangePassword.java	High
Static IV in AES-CBC Encryption	CryptoClass.java	Medium
Sensitive Financial Data Exposure Through Logs	DoTransfer.java	Medium
Insecure WebView Loading of Local Transaction Statements	ViewStatement.java	Medium
Exported Content Provider Without Access Control	TrackUserContentProvider.java	Medium
Phone Number Exposure Through Logs	ChangePassword.java	Low
Easily Bypassable Root Detection	PostLogin.java	Low

Lab Setup - InsecureBankv2 APK

Command:

mkdir -p ~/Mobile-Pentesting/APKs
mkdir -p ~/Mobile-Pentesting/Extracted
cd ~/Mobile-Pentesting/APKs
Clone the InsecureBankv2 repository:
git clone https://github.com/dineshshetty/Android-InsecureBankv2.git
Enter the project directory:
cd Android-InsecureBankv2
Check the downloaded files:
ls -lah
Verify the APK:
file InsecureBankv2.apk
Calculate the SHA-256 hash:
sha256sum InsecureBankv2.apk

- This is the initial setup and APK reconnaissance performed in Kali Linux. The Mobile-Pentesting/APKs workspace was prepared, the InsecureBankv2 project/APK was located, and the InsecureBankv2.apk file was verified using the file command as an Android package (APK). This confirms that the correct APK was available and ready for further static analysis.

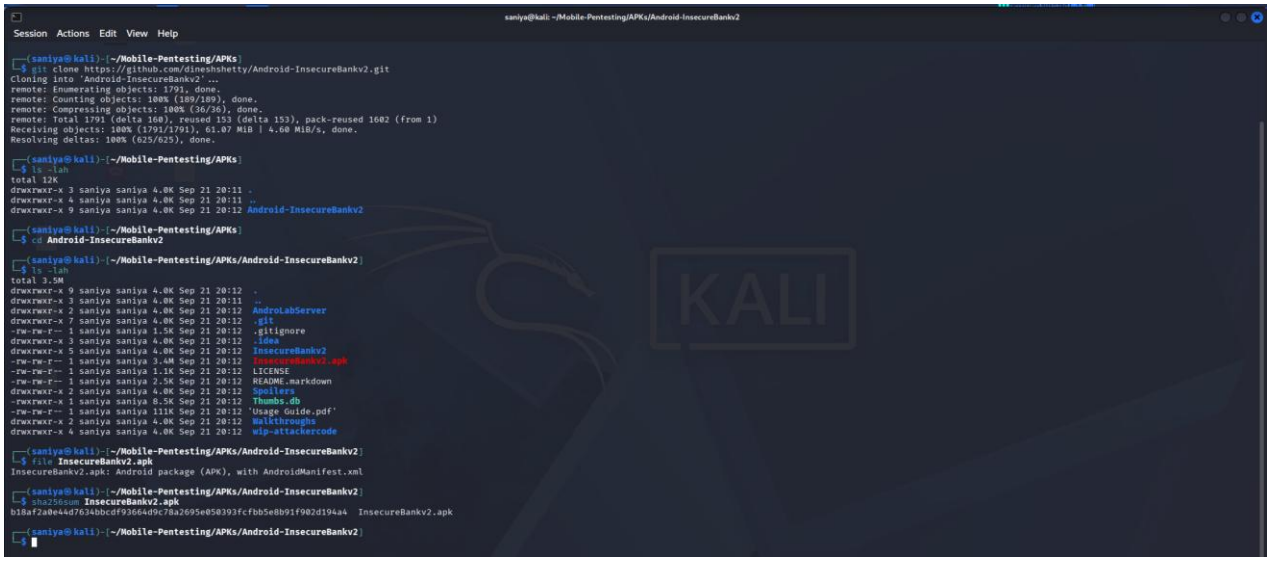


Figure 1 – Verification of the InsecureBankv2 APK in the Kali Linux Analysis Workspace

Step 1 - APK Reconnaissance

Objective

Determine what information can be learned about the InsecureBankv2 APK before decompiling it.

Command: cd ~/Mobile-Pentesting/APKs/Android-InsecureBankv2

Run: aapt dump badging InsecureBankv2.apk

Grep command: aapt dump badging InsecureBankv2.apk | grep -E "package:|sdkVersion|targetSdkVersion|uses-permission:|application-debuggable|launchable-activity:"

- The filtered AAPT reconnaissance output provides a concise overview of the InsecureBankv2 application's security-relevant configuration. The application requests multiple sensitive permissions and is marked as debuggable. The LoginActivity is identified as the launchable activity. These observations provide an initial understanding of the application's attack surface and identify areas requiring further analysis in the AndroidManifest.xml and source-code review. The presence of permissions or the debuggable flag alone does not confirm a vulnerability.



Figure 2 – Filtered APK Reconnaissance Using AAPT

Key findings:

- Package name: com.android.insecurebankv2
- Version: 1.0
- Minimum SDK: 15
- Target SDK: 22
- Multiple sensitive permissions are requested, including Internet, external storage, SMS, contacts, phone state, call logs, accounts, and location.
- The APK is marked as application-debuggable.

- The launchable activity is com.android.insecurebankv2.LoginActivity.

Step 2 - Understand APK Structure

An APK is essentially a specially structured ZIP archive containing compiled application code, resources, configuration files, native libraries and signing information.

Inspect and Extract

Run:

cd ~/Mobile-Pentesting/APKs/Android-InsecureBankv2

unzip -l InsecureBankv2.apk | head -30

Create the extraction directory:

mkdir -p ~/Mobile-Pentesting/Extracted/InsecureBankv2-Raw

Extract the APK:

unzip InsecureBankv2.apk -d ~/Mobile-Pentesting/Extracted/InsecureBankv2-Raw

Enter the extracted directory:

cd ~/Mobile-Pentesting/Extracted/InsecureBankv2-Raw

List the extracted files:

ls -lah

- The APK was inspected as a structured ZIP archive to understand its internal components before performing detailed static analysis. The listing shows the Android manifest, compiled application code (classes.dex), application resources (res/ and resources.arsc), and signing-related files under META-INF/. These components provide the basis for subsequent Manifest, resource, signing, JADX, APKTool, and Smali analysis.



Figure 3 – InsecureBankv2 APK Internal Structure and Extracted Files

After finding InsecureBankv2.APK I was extracted into a dedicated analysis directory using the unzip command. The extracted contents include application resources, layouts, images, XML configuration files, and other files packaged within the APK. This extraction provides access to the application's internal structure for the next stages of static security analysis

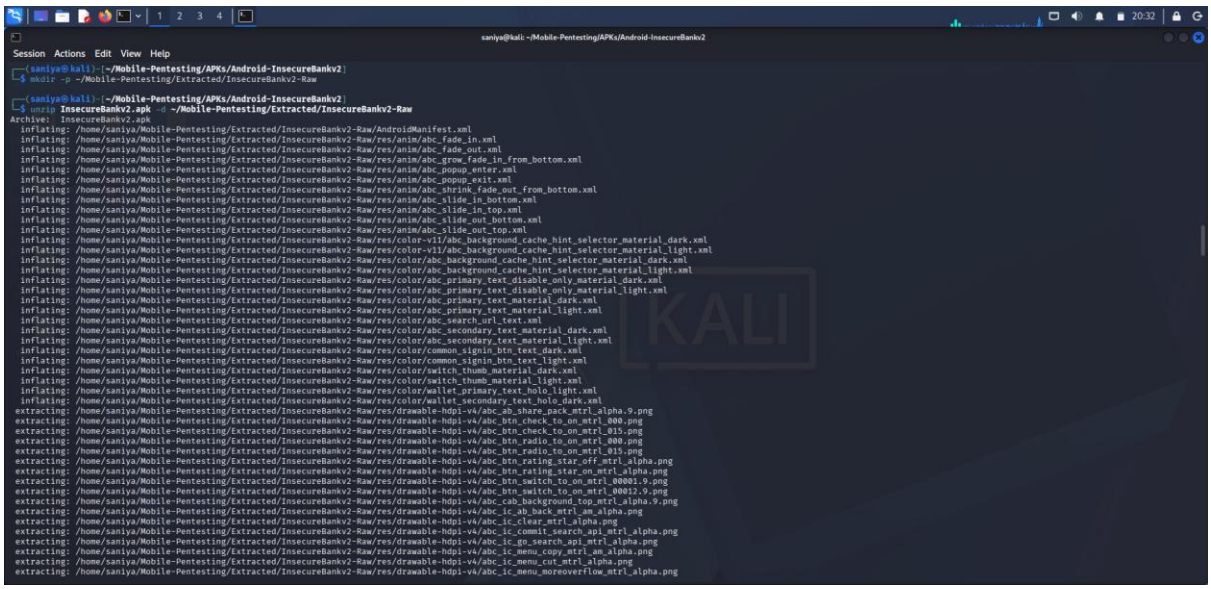


Figure 4.A – InsecureBankv2 APK Extraction

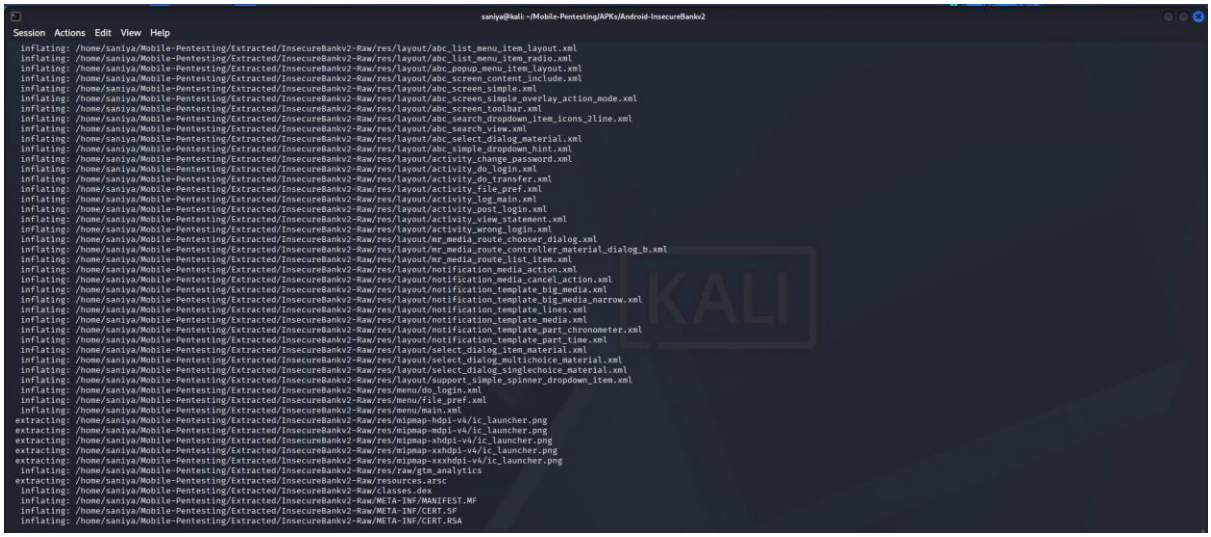


Figure 4.B – InsecureBankv2 APK Extraction

- After extracting the InsecureBankv2 APK, the extracted files were checked using the ls -lah command. The directory contains important APK components such as AndroidManifest.xml, classes.dex, META-INF, res, and resources.arsc. These files provide the basic structure required for further analysis of the application's configuration, source code, resources, and signing information.

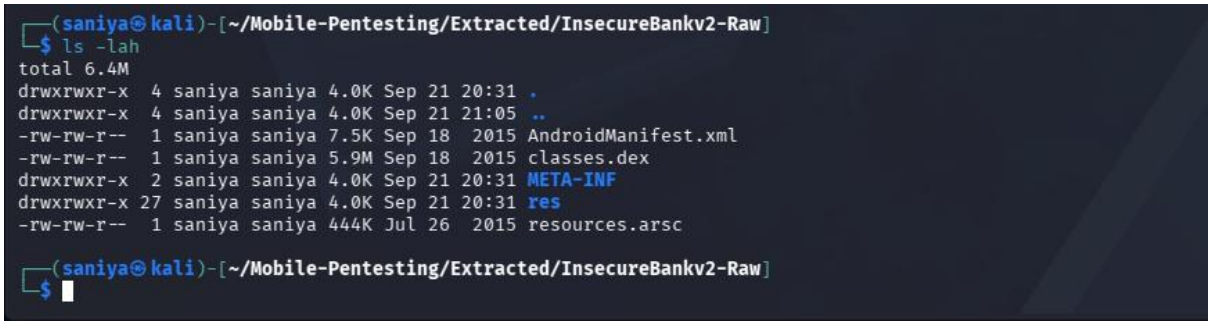


Figure 5 – InsecureBankv2 Extracted APK Directory

Readable String Reconnaissance
strings classes.dex | head -30
strings classes.dex | grep -iE 'password|secret|key|http'
command: strings classes.dex | grep -iE 'super secret|superSecurePassword|rememberme_password|decryptedPassword|password is:|Updated Password|Change Password|/changepassword|/fetchHttpRequest|/httpTrack'

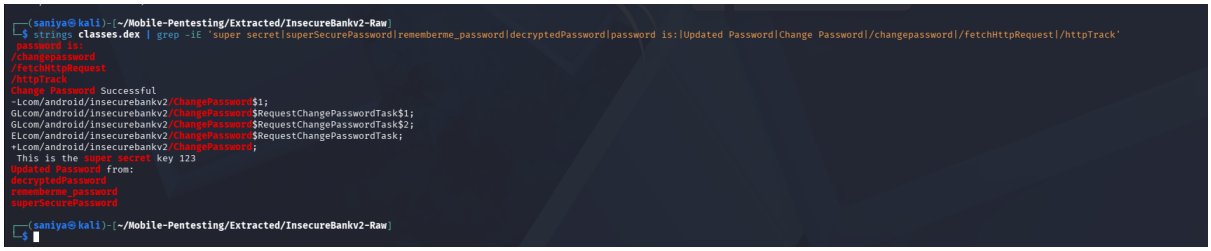


Figure 6 – Security-Relevant Strings Identified in InsecureBankv2 classes.dex

Finding	Evidence from Output	Significance
Password-related data	password is:	Indicates password-related processing
Password change	/changepassword	Shows password-change functionality
HTTP-related functionality	/fetchHttpRequest, /httpTrack	Indicates HTTP request/tracking functionality
Password change result	Change Password Successful	Confirms password-change operation
Hardcoded cryptographic key	This is the super secret key 123	Indicates a hardcoded key embedded in the APK
Password decryption	decryptedPassword	Indicates password decryption logic
Remembered password	rememberme_password	Indicates credential-related storage
Stored protected password	superSecurePassword	Indicates password storage/encryption-related logic
Password update message	Updated Password from:	Indicates password-update processing

Step 3 - AndroidManifest.xml Analysis
The AndroidManifest.xml extracted directly from the APK is binary XML. Decode it with APKTool before reviewing the configuration.

Command: cd ~/Mobile-Pentesting/Extracted/InsecureBankv2-Apktool
Command: cat AndroidManifest.xml

Grep command: grep -nE 'uses-permission|debuggable|allowBackup|activity|provider|receiver|service|exported' AndroidManifest.x

AndroidManifest.xml Analysis:

The decoded AndroidManifest.xml was examined to identify security-related application **configurations, permissions,** and **exported** components. The application is configured with **android:debuggable="true"** and **android:allowBackup="true".** It requests several sensitive permissions, including SMS, contacts, call-log, account, phone-state, storage, and location permissions. Multiple application components are explicitly exported, including PostLogin, DoTransfer, ViewStatement, ChangePassword, TrackUserContentProvider, and MyBroadCastReceiver. These exported components require further source-code analysis to determine whether unauthorized interaction is possible.

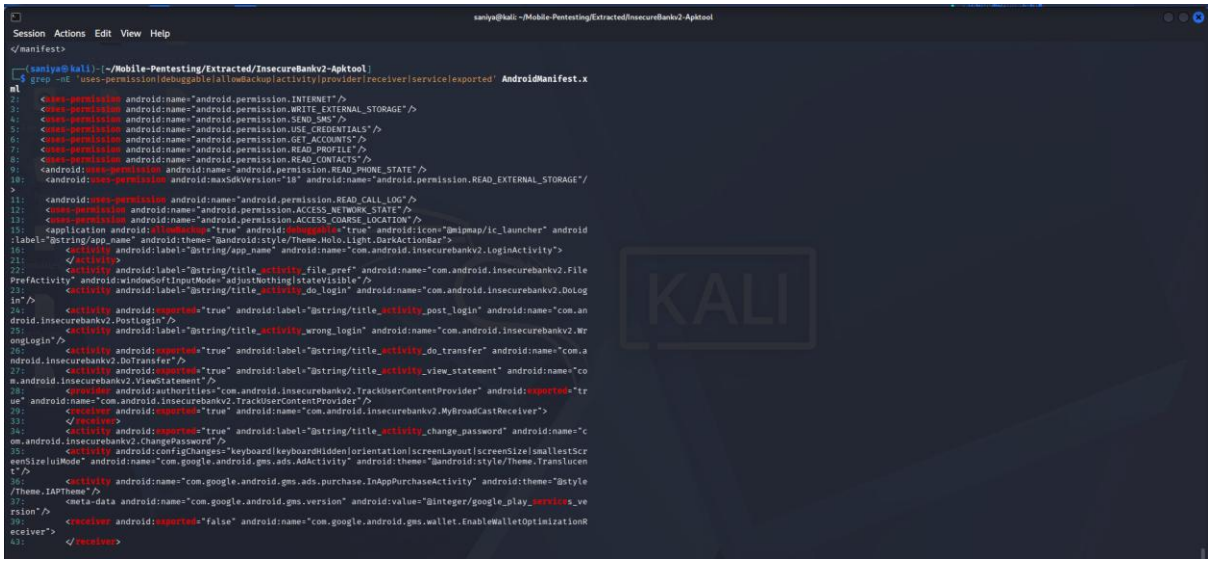


Figure 7 – InsecureBankv2 AndroidManifest.xml Security Analysis

Step 4 – Analyze res/ and Resources

The res/ directory contains application resources such as strings, XML files, layouts, images, and configuration files. In InsecureBankv2, these resources can reveal application behavior and may contain security-sensitive information such as endpoints, tokens, passwords, or configuration values.

Commands: find res -maxdepth 2 -type d

grep command: grep -RniE 'password|api.?key|secret|token|server_ip|server_port|database|admin' res/ | grep -vE 'public.xml|xmlns|google|abc_|common_'



Figure 8 – InsecureBankv2 Resource Analysis and Security-Relevant Configuration

Finding	Evidence	What it tells us
Admin setting	is_admin="no"	An admin-related configuration exists
Password handling	loginscreen_password	Login password field exists
Server configuration	server_ip, server_port	The app has configurable server connection fields
Password change	ChangePassword	Password-change functionality exists
New password field	editText_newPassword	New password input exists
Password masking	android:password="true"	Password input is masked in the UI
Change-password button	button_newPasswordSubmit	Password-change operation is exposed in the UI

Step 5 – Analyze Native Libraries (lib/)

The lib/ directory contains compiled native libraries, normally .so files written in C/C++. These libraries can contain security-sensitive logic, hardcoded secrets, cryptographic material, URLs, or JNI/native authentication code.

Command: cd ~/Mobile-Pentesting/Extracted/InsecureBankv2-Apktool

ls -lah

- The extracted InsecureBankv2 APK was examined for native libraries. The extracted directory contains AndroidManifest.xml, assets, classes.dex, META-INF, res, and resources.arsc, but no lib/ directory or .so native libraries were identified. Therefore, native-code analysis for hardcoded secrets, native

cryptographic logic, JNI functionality, and unsafe C/C++ functions is not applicable to this APK.

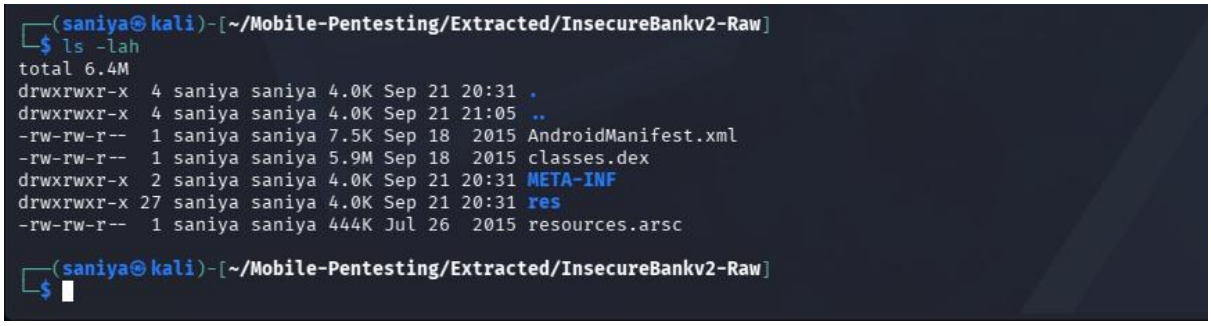


Figure 9 – InsecureBankv2 Native Library Analysis

Step 6 – Analyze META-INF/ and APK Signing

META-INF/ contains APK signing-related files, especially for the older **JAR-style (v1) signing mechanism.**

Command:

cd ~/Mobile-Pentesting/Extracted/InsecureBankv2-Raw
find META-INF -type f
ls -lah META-INF

- The META-INF directory of the extracted InsecureBankv2 APK was examined to identify files associated with the APK's JAR-style signing information. The directory contains MANIFEST.MF, CERT.RSA, and CERT.SF, which are the relevant signing-related files present in the APK.
-

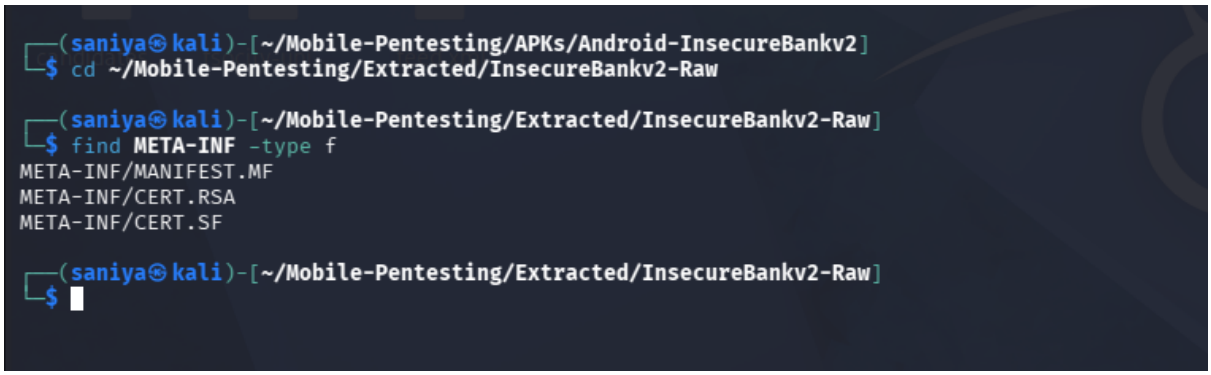


Figure 10 – InsecureBankv2 META-INF APK Signing Files

Finding:

Security Check	Finding
Signature verification	PASS
Number of signers	1
v1 / JAR signing	YES
v2 signing	NO
v3 signing	NO
v3.1 signing	NO
v4 signing	NO
SourceStamp	NO
Signer DN	CN=Dinesh Shetty, OU=Services, O=SI, L=Boston, ST=MA
SHA-256	8092db81ae717486631a1534977def465ee112903e1553d38d41df8abd57a375
SHA-1	a1bae91a2b1620f6c9dab425e69fc32ba1e97741
MD5	6a736d89abb13d7165e7cff905ac928d

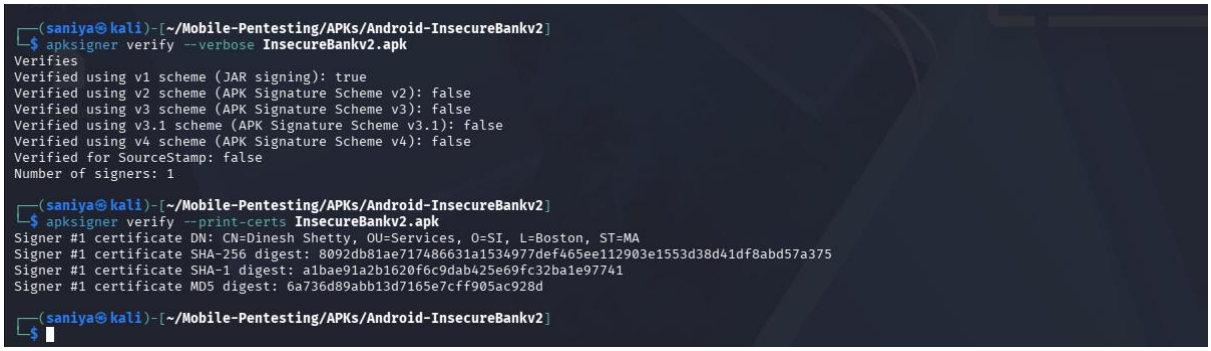


Figure 11 – InsecureBankv2 APK Signature Verification and Certificate Analysis

Step 7 – JADX Decompilation

JADX reconstructs readable Java-like source code from the APK's DEX bytecode. This is useful because we can inspect the application's actual logic and verify whether the security issues identified from the Manifest, resources, and strings are really used in the code.

Command:

jadx -d ~/Mobile-Pentesting/Extracted/InsecureBankv2-JADX ~/Mobile-Pentesting/APKs/Android-InsecureBankv2/InsecureBankv2.apk

ls -lah



Figure 12 – InsecureBankv2 JADX Decompilation Output

Step 8 – Manual Source-Code SAST

Now that JADX has successfully decompiled InsecureBankv2, we can perform manual Static Application Security Testing (SAST).

Command:

cd ~/Mobile-Pentesting/Extracted/InsecureBankv2-JADX/sources/com/android/insecurebankv2

Then:

Ls -lah

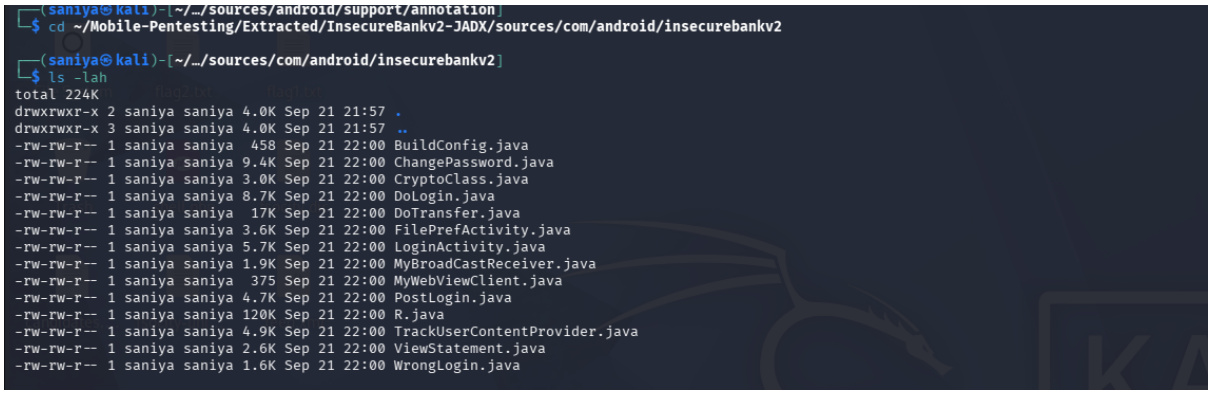


Figure 13 – InsecureBankv2 Decompiled Java Source Files

8.1: ChangePassword.java

Command: cat ChangePassword.java

grep command: grep -nE 'protocol =|HttpPost|newpassword|putExtra|sendBroadcast|System\.out\.println|intent\.getStringExtra' ChangePassword.java



Figure 8.1 – InsecureBankv2 Cleartext Communication and Sensitive Data Exposure

1. Cleartext Password Transmission

The application uses HTTP instead of HTTPS when changing the password. The new password is sent to the /changepassword server endpoint. Because HTTP is not encrypted, an attacker who can monitor the network may be able to see the username and new password.

2. Password Exposed Through Broadcast

After changing the password, the application places the new password inside a broadcast Intent and sends it using sendBroadcast(). Sensitive information such as passwords should not be shared through general broadcasts because other components may potentially receive the data.

3. Phone Number Exposed in Logs

The application prints the user's phone number to the application log using System.out.println(). Sensitive personal information in logs can create a privacy risk if the logs are accessible.

4. Potential Authorization Issue

The application gets the username from an Intent and uses it in the password-change request. The code shown does not perform an authorization check before sending the request. This is a potential issue and requires runtime/server-side testing to confirm whether another user's password can actually be changed.

8.2: CryptoClass.java

Command: cat CryptoClass.java

grep command: grep -nE 'String key|ivBytes|Cipher.getInstance|SecretKeySpec|aes256encrypt|aes256decrypt' CryptoClass.java



Figure 8.2 – InsecureBankv2 Hardcoded Encryption Key and Static IV

1. Hardcoded Encryption Key

22: String key = "This is the super secret key 123";

The AES encryption key is stored directly inside the application. Anyone who obtains and decompiles the APK can recover the key and potentially decrypt protected data.

Finding: Hardcoded Cryptographic Key
Risk: High

2. Static IV in AES-CBC

23: byte[] ivBytes = {0, 0, 0, ...};

28: Cipher.getInstance("AES/CBC/PKCS5Padding");

The application uses an all-zero, fixed IV with AES-CBC. The same IV is reused for encryption instead of generating a new unpredictable IV for each encryption operation.

8.3: DoLogin.java

Command: cat DoLogin.java

grep command: grep -nE 'saveCreds|SharedPreferences|superSecurePassword|aesEncryptedString|putString|commit' DoLogin.java

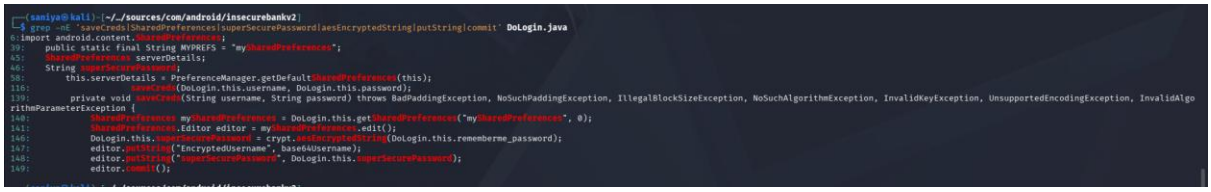


Figure 8.3 – InsecureBankv2 Credential Storage in SharedPreferences

The application stores remembered login credentials in local SharedPreferences. Although the password is encrypted before storage, the encryption key is hardcoded in the APK and the encryption uses a static IV. Therefore, an attacker who obtains the APK and the stored preferences may be able to recover the stored password.

8.4: DoTransfer.java

Command: cat DoTransfer.java

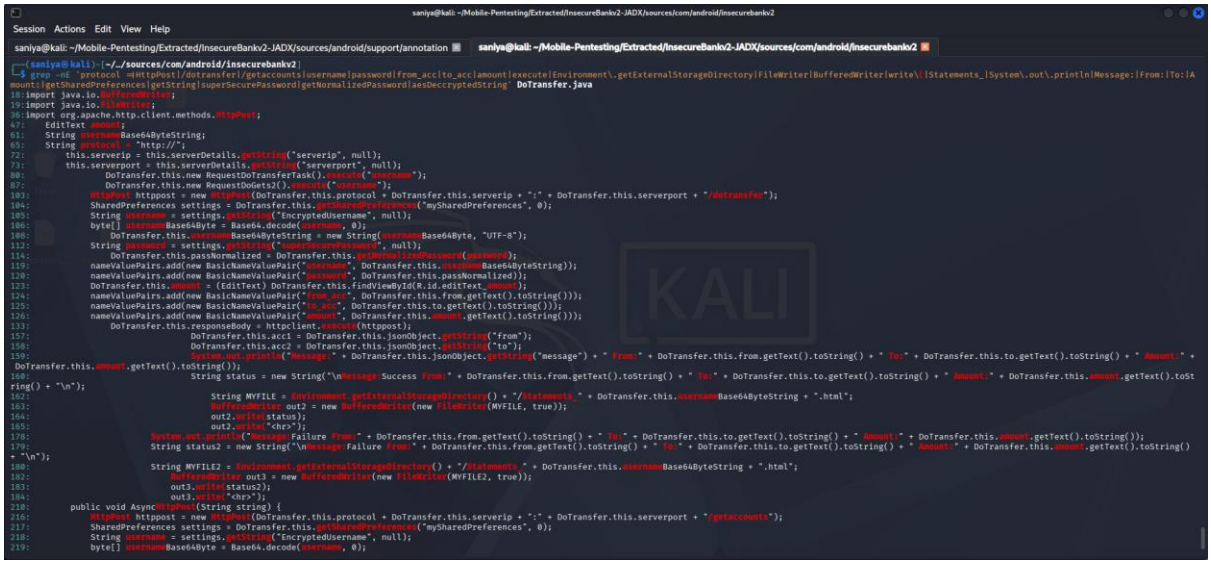


Figure 8.4 – InsecureBankv2 Sensitive Data Transmission, Storage, and Logging

No.	Vulnerability / Finding	Evidence	Impact
1	**Cleartext Transmission of Sensitive Data**	http:// is used, and username, password, source account, destination account, and amount are sent to /dotransfer.	Sensitive credentials and financial transaction data could be intercepted over the network.
2	**Insecure Storage of Financial Data**	Transaction details are written to files in external storage using FileWriter.	Sensitive account and transaction information may be exposed through local storage.
3	**Sensitive Data Exposure Through Logs**	Account numbers and transaction amount are printed using System.out.println().	Financial information may be exposed through accessible application/device logs.

8.5 : LoginActivity.java

Command: cat LoginActivity.java

grep command: grep -nE 'getSharedPreferences|getString|superSecurePassword|aesDeccryptedString|decryptedPassword|setText' LoginActivity.java

```
┌──(saniya㉿kali)-[~/…/sources/com/android/insecurebankv2]
└─$ grep -nE 'getSharedPreferences|getString|superSecurePassword|aesDeccryptedString|decryptedPassword|setText' LoginActivity.java
36:         String mess = getResources().getString(R.string.is_admin);
73:         SharedPreferences settings = getSharedPreferences("mySharedPreferences", 0);
74:         String username = settings.getString("EncryptedUsername", null);
75:         String password = settings.getString("superSecurePassword", null);
85:             this.Username_Text.setText(this.usernameBase64ByteString);
87:             String decryptedPassword = crypt.aesDeccryptedString(password);
88:             this.Password_Text.setText(decryptedPassword);

┌──(saniya㉿kali)-[~/…/sources/com/android/insecurebankv2]
└─$
```

Figure 8.5 – Analyze LoginActivity.java file

8.6: MyBroadCastReceiver.java

Command: cat MyBroadCastReceiver.java

grep command: grep -nE 'onReceive|getStringExtra|superSecurePassword|aesDeccryptedString|decryptedPassword|newpass|textMessage|System\.out\.println|SmsManager|sendTextMessage' MyBroadCastReceiver.java

```
┌──(saniya㉿kali)-[~/…/sources/com/android/insecurebankv2]
└─$ grep -nE 'onReceive|getStringExtra|superSecurePassword|aesDeccryptedString|decryptedPassword|newpass|textMessage|System\.out\.println|SmsManager|sendTextMessage' MyBroadCastReceiver.java
7:import android.telephony.SmsManager;
16:    public void onReceive(Context context, Intent intent) {
17:        String phn = intent.getStringExtra("phonenumber");
18:        String newpass = intent.getStringExtra("newpass");
25:                String password = settings.getString("superSecurePassword", null);
27:                String decryptedPassword = crypt.aesDeccryptedString(password);
29:                String textMessage = "Updated Password from: " + decryptedPassword + " to: " + newpass;
30:                SmsManager smsManager = SmsManager.getDefault();
31:                System.out.println("For the changepassword - phonenumber: " + textPhoneno + " password is: " + textMessage);
32:                smsManager.sendTextMessage(textPhoneno, null, textMessage, null, null);
39:        System.out.println("Phone number is null");

┌──(saniya㉿kali)-[~/…/sources/com/android/insecurebankv2]
└─$
```

Figure 8.6 – InsecureBankv2 Password Disclosure Through SMS and Logs

Findings – MyBroadCastReceiver.java

No.	Finding	Evidence
1	**Password Disclosure Through SMS**	Lines 27–32 decrypt the stored password, combine it with the new password, and send both through SmsManager.sendTextMessage().
2	**Password Disclosure Through Logs**	Line 31 prints the complete password message using System.out.println().

8.7: ViewStatement.java

Command: cat ViewStatement.java

grep command: grep -nE 'getIntent|getStringExtra|Environment\.getExternalStorageDirectory|loadUrl|setJavaScriptEnabled|setSaveFormData|WebView|file://' ViewStatement.java



Figure 8.7 – InsecureBankv2 Insecure WebView Loading of Sensitive Transaction Statements

The application loads transaction statements stored on external storage into a WebView using a file:// URL and enables JavaScript. This insecure local-file handling may expose sensitive financial information.

8.8 : TrackUserContentProvider

Command: cat TrackUserContentProvider.java

grep command: grep -nE 'extends ContentProvider|public Cursor query|public Uri insert|public int update|public int delete|this\.db\.(query|insert|update|delete)|checkCallingPermission|enforceCallingPermission' TrackUserContentProvider.java



Figure 8.8 – Exported Content Provider Without Access Control

The TrackUserContentProvider exposes database query, insert, update, and delete operations. The code does not show any permission check such as checkCallingPermission() or enforceCallingPermission(). Because the provider is exported, another application may potentially access or modify the provider's database records.

Step 9 – APKTool + Smali Analysis

The output shows the APKTool-decoded smali directory and the .smali files generated from the InsecureBankv2 APK. It includes application files such as PostLogin.smali, CryptoClass.smali, LoginActivity.smali, ChangePassword.smali, DoLogin.smali, DoTransfer.smali, ViewStatement.smali, MyBroadCastReceiver.smali, and TrackUserContentProvider.smali. It also shows Android framework/support Smali files under smali/android/…. The find command was used to enumerate the available Smali files for low-level APK analysis.

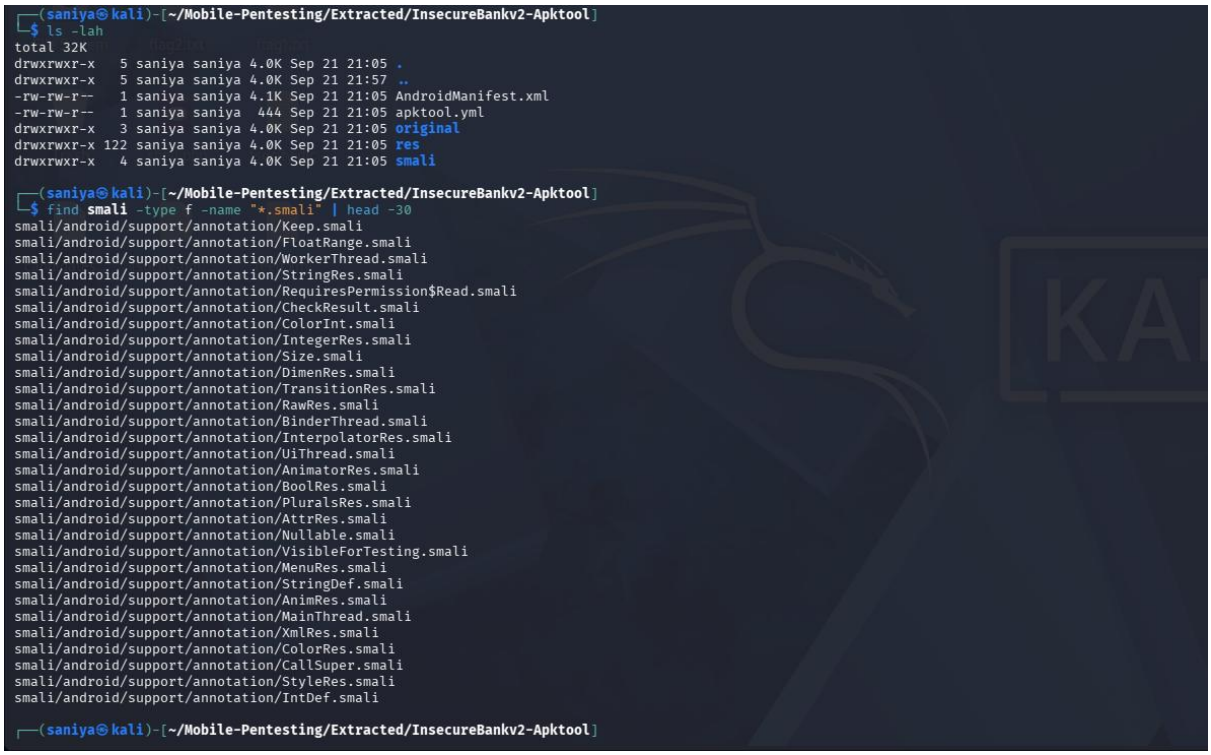


Figure 9 – APKTool Smali Directory Analysis

9.1: CryptoClass.smali

Command: grep -nE 'const-string|ivBytes|AES|CBC|PKCS5Padding|SecretKeySpec|IvParameterSpec|Cipher;->getInstance' smali/com/android/insecurebankv2/CryptoClass.smali

The Smali code shows that the AES encryption key **"This is the super secret key 123"** is hardcoded inside the application. It also shows the use of a stored ivBytes value with **AES/CBC/PKCS5Padding** for encryption and decryption. Because the key and IV are embedded in the application, an attacker who obtains the APK can recover these values and potentially decrypt protected data.

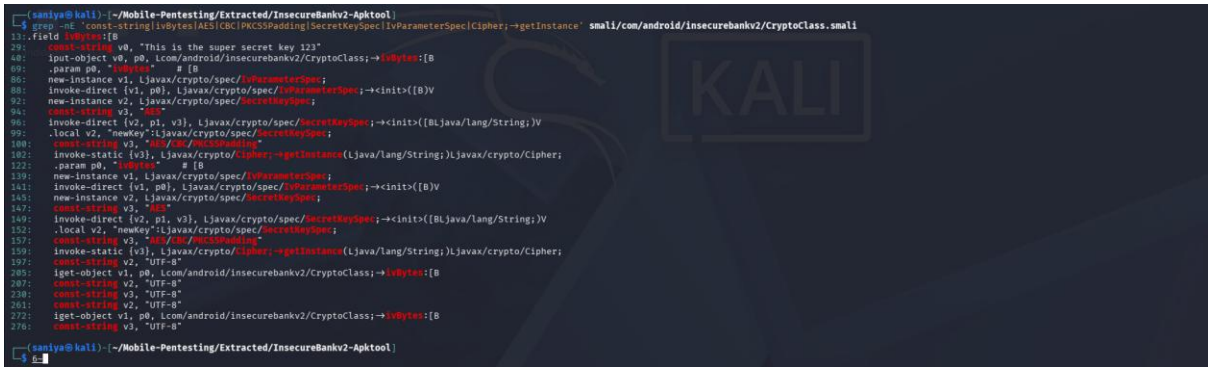


Figure 9.1 – Smali Evidence of Hardcoded Cryptographic Key and Static IV

9.2 – Root Detection Analysis in Smali Code



Figure 9.2 – InsecureBankv2 Root Detection Security Control

Finding

Security Control: Easily Bypassable Root Detection

The root-detection logic is implemented within the application itself and relies on simple checks for known root indicators. Such client-side checks can potentially be bypassed through application modification or runtime hooking.

Evidence:

- doesSuperuserApkExist("/system/app/Superuser.apk")
- doesSUexist()
- Runtime.getRuntime().exec(... "/system/xbin/which", "su")

Remediation

The following remediation measures are recommended based on the security findings identified during the static analysis:

1. **Use HTTPS for all network communication**
   Replace HTTP endpoints with HTTPS and enforce secure TLS communication to protect usernames, passwords, account numbers, and transaction amounts from interception.

2. **Remove hardcoded cryptographic keys**
   Cryptographic keys should not be stored directly in the APK. Use the **Android Keystore** or another secure key-management mechanism.

3. **Use a secure encryption mode and unique IVs**
   Replace AES-CBC with a modern authenticated encryption mode such as AES-GCM. Generate a unique, unpredictable IV/nonce for every encryption operation.

4. **Improve credential storage**
   Avoid storing reusable passwords in SharedPreferences. If credentials must be remembered, use secure Android storage and appropriate authentication/session mechanisms.

5. **Do not send passwords through broadcasts or SMS**
   Passwords and other authentication secrets should never be placed in general broadcast Intents or SMS messages. Use secure internal communication and avoid transmitting plaintext credentials.

6. **Remove sensitive information from logs**
   Do not log passwords, phone numbers, account numbers, transaction amounts, or other sensitive information. Production builds should also disable unnecessary debugging output.

7. **Protect financial transaction data**
   Avoid storing sensitive transaction information in shared/external storage. Use protected application-private storage with appropriate access controls.

8. **Secure WebView usage**
   Avoid loading sensitive local files through file:// URLs. Restrict WebView access to only the required resources and avoid enabling JavaScript unless it is necessary.

9. **Restrict Content Provider access**
   The exported TrackUserContentProvider should not be publicly accessible unless required. If it must be exported, enforce appropriate permissions and caller authorization before allowing query, insert, update, or delete operations.

10. **Strengthen root-detection controls**
    Root detection should not be treated as a strong security boundary. Security-sensitive authorization decisions should be enforced on the server rather than relying only on client-side root checks.

11. **Review authorization of password-change operations**
    The application should verify that the authenticated user is authorized to change the specified account's password rather than trusting a username received through an Intent. The report identifies this as a potential issue requiring runtime/server-side confirmation.

Conclusion

The static security assessment of **InsecureBankv2** examined the APK from multiple levels, including APK structure, Android Manifest, resources, signing information, JADX-decompiled Java source, and APKTool-generated Smali code. The analysis identified weaknesses involving **cleartext communication, cryptographic key management, credential protection, sensitive-data exposure, local storage, WebView configuration, exported components, Content Provider access control, and client-side root detection**.

The assessment demonstrates that sensitive security logic and data-handling practices can be identified directly from an Android APK through static analysis. JADX provided readable Java-level evidence, while APKTool and Smali analysis provided lower-level confirmation of selected implementation details, including the root-detection routine.

Overall, the findings show the importance of **secure communication, proper cryptographic key management, protected credential handling, least-privilege component exposure, secure local data handling, and server-side enforcement of security decisions** when developing Android applications.